



ÖZYEĞİN UNIVERSITY
FACULTY OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE

CS 402

2023 Spring

SENIOR PROJECT REPORT

**A Fintech API for the conversion and management of SWIFT MT
financial
Messages to modern standards**

By
Maram Elsebakhi & Emir Aydin

Supervised By
Asst. Prof. Dr. Ismail Ari

Declaration of Own Work Statement/ (Plagiarism Statement)

Hereby I confirm that all this work is original and my own. I have clearly referenced/listed all sources as appropriate and given the sources of all pictures, data etc. that are not my own. I have not made any use of the essay(s) or other work of any other student(s) either past or present, at this or any other educational institution. I also declare that this project has not previously been submitted for assessment in any other course, degree or qualification at this or any other educational institution.

**Student Name and
Surname:**

Emir Aydin Maram Elsebakhi

Signature:

Place, Date:

Istanbul, Turkey – 02.06.2023

Abstract

In this project, we aim to develop modern a Fintech solution that is both legacy-complaint and adheres to modern PSD2 banking standards. For the legacy part, we gathered sample SWIFT MT messages and converted them to JSON files to make them compatible with SQL and NoSQL database systems as well as Open Banking API. This problem was motivated by Movetech company, which indicated a need for directly exchanging SWIFT MT messages among existing customers and banks. Our solution can also make HTTP REST calls to Open Banking API to collect bank, customer or account related information. If needed, this information can be converted to SWIFT MT messages for legacy transactions and support.

To implement our project, we used Java language, Spring framework and Maven project management. As NoSQL we used document-based MongoDB. For frontend, we used MongoDB Compass GUI, and we could use Postman to manage APIs and cURL to send HTTP GET/POST messages to the server for getting responses in JSON format as data. We also utilized an open-source SWIFT MT converter API by Prowide Core company. The converted versions of MTs are created as JSON files. We also tested writing these messages to Amazon AWS to store them in a cloud database. Finally, we did time measurement of the code and reported results.

Contents

I.	Introduction	5
II.	Background.....	6
III.	Problem Statement	7
IV.	Solution Approach	8
1.	SWIFT MT.....	8
2.	Provide Core API.....	9
3.	Build tool and database: Maven and MongoDB.....	10
4.	Spring MVC.....	11
5.	RESTful services	16
6.	Banking APIs	17
V.	Results and Discussion.....	21
VI.	Related Work	24
VII.	Conclusion and Future Work	26
	Acknowledgements.....	27
	References.....	28
	Appendix.....	30

I. Introduction

In the background section, we described the industry standards and tools we need for our project, namely the open banking payment service directives, SWIFT messages, the open-source API Provide Core, the build tool Maven, and the database tool MongoDB. In the problem statement section, we described the purpose of the project and the requirements by the company Movetech. We introduced the concept of Open Banking in our project and our plan to create an API that is standardized with PSD2 regulation to create an industrial solution. Afterwards, in the Solution approach section we described our solution and design decisions for integrating an old banking standard which is SWIFT MT into a new Fintech solution by converting the SWIFT MT fin files into JSON files and storing them in a database, while doing so we explained and broke down the meaning and details of SWIFT MT messages and the use case of the Provide Core API. Then we explained which database is used to store those JSON files. Thus, we mentioned RESTful services to explain the software architectural design while describing our web application which was built by using Spring MVC framework. We briefly described more implementation details such as the knowledge used, functionality of the project, design of the code, limitations in our approach, and tools we have used and the engineering standards. Next, in the results section we shared the result of the current version of the project, and we discussed it with time measurement in the results and discussion section alongside the testing results of the Banking APIs. In the related works section, we also shared the approaches and works that are related to our project scope. As we conclude our current version of the project, our future goals to reach the planned version of the project are described while we are explaining the ethical issues of the project in the Conclusion and Future Work section.

II. Background

Society for Worldwide Interbank Financial Telecommunications (SWIFT) was founded in the 1970s. In our project, SWIFT message types are used as a standardized format for the electronic exchange of financial messages among banks and other financial institutions. SWIFT message types are used for money transfer, the exchange of trade information and settlement of financial transactions [1]. By including International Bank Account Number (IBAN), which is used to uniquely identify bank accounts in international transactions, in SWIFT MT messages helps to identify the exact recipient account and avoid potential errors or delays in the payment process. Bank Identifier Code (BIC), which is used to identify branch codes of banks [2], is the letter combination that generates SWIFT code. The Payment Service Directive 2 (PSD2) which began in 2016 is a directive of the European Union (EU) was updated version of PSD from 2008. It regulates account service payment service providers (ASPSP) or “banks” as well as 3-rd Party Providers (TPP) or “fintech companies” in the EU. Some of the key provisions that PSD2 includes are open banking, in which banks make certain data and functionalities available for TPP through APIs that allows them to compete with banks, and strong customer authentication that payment service providers (PSP) implement strong authentication measures to verify the identity of customers [3]. It is predicted that PSD3, which is more focused authentication and fighting with fraud, will be announced within 3 years.

The importance of legacy compliance and interoperability has always been an important problem of the financial sector. Nikolov, et al. (Interoperability of bank statements: A case study, 2015) [4], shared a case study about integrating a user’s bank accounts and showed the difficulty of interoperability due to different formats (XML, text, SWIFT MT, etc.) used by different financial institutions. SWIFT to Open Banking transition has recently been motivated in the literature Madhavi, et al. (A Novel Bank Statements Reconciliation Using Message Transfer940 Parser, 2017) [5] issued the parsing the MT940 messages helps to decode the SWIFT formatted bank statements to a normal XML format. By using the adapter, the interoperability between devices is maintained and the resultant data can be retrieved in the form of XML format.

Our work extends these efforts by adding support for modern Open Banking API that are PSD2 compliant.

III. Problem Statement

Our project emerged from a request that we have received from Movetech, which is a software company that serves logistic companies with its own ERP program by managing their day-to-day businesses including shipment tracking, accounting, and position expansion. It is in ITU ARI Teknokent. The company aims to provide Industry 4.0 or Logistic 4.0 by increasing efficiency and transparency, improving the supply chain, and increasing information and communication technologies with decentralized decision-making structures.

The request that we received was about assisting them by providing a service to prevent them from being dependent on the APIs which belong to banks or other third-party providers (TPPs). They expected us to integrate SWIFT MT940 messages with banks to this out-source service by gathering the necessary information from their customers as well. Apart from SWIFT's traditionally direct connection with the bank and network authentication, PSD2 provides the usage of third-party tools with your bank's information and for not just an individual one by peer to peer (P2P) connection-based authentication and standardized APIs [18]. By PSD2 regulation, we aim to develop an API that satisfies the needs of the company while making it standardized based on industry standards. For this purpose, we suggest using PSD2 regulation in payment services by using one of their provisions, which is Open Banking as an approach to Fintech. Through Open Banking APIs, certain data or functionality is made available by banks to third-party providers through the APIs. This allows TPPs to offer new payment services such as we plan to develop for Movetech company. Furthermore, being in interaction with different banking APIs may cause unnecessary costs and time consumption. Implementing an API prevents the company from those consequences. Before the implementation of the API, MTs should be integrated as converting them into JSON files to achieve a more readable structure for applications. For that reason, we decided to implement a converter by using the Provide Core API for using its tools to manipulate and easily parse the SWIFT MT files. After the conversion, JSON files should be stored in a database. By using MongoDB as a NoSQL database, we aim to easily set it up and use its flexibility and storage of a wide variety of data. Handling distributions of data in multiple servers or locations makes NoSQL databases well-suited for cloud-based approaches.

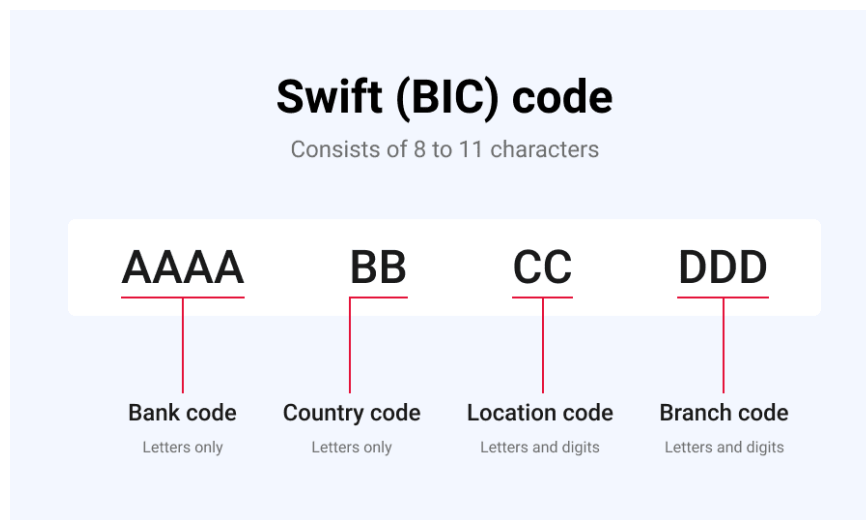
IV. Solution Approach

1. SWIFT MT

For the first part of our project, we focused on implementing a SWIFT MT converter tool to satisfy the initial customer request of integrating the customers who still require the use of legacy banking protocols such as the SWIFT MT940. To start off we researched into the SWIFT MT standard, its history, use case and message types. Then we needed an understanding of the format of the messages themselves.

First, we define SWIFT (Society for Worldwide Interbank financial Telecommunications) as the bank code for sending international payments [2]. SWIFT network is only the messaging network that communicates transaction orders between financial institutions.

Next, we define SWIFT messages and code, the SWIFT code is a combination of letters known as BIC (bank identifier code) that is the branch code of banks.



[2]

Banks need to be SWIFT members and receive a SWIFT code to be part of the global SWIFT network. This unique swift code will be used in all transaction types over this network later.

SWIFT messages have various message types which are codenamed MT, which is where the SWIFT MT naming is from, the following table showcases the different types:

Category	Message type	Description	Number of message types
0	MT0xx	System messages	-
1	MT1xx	Customer payments and cheques	19
2	MT2xx	Financial institution transfers	18
3	MT3xx	Treasury markets	27
4	MT4xx	Collection and cash letters	17
5	MT5xx	Securities Markets	60
6	MT6xx	Treasury markets – metals and syndications	22
7	MT7xx	Documentary credits and guarantees	29
8	MT8xx	Traveller's cheques	11
9	MT9xx	Cash management and customer status	21

[2]

In our project, since the customer requested specifically that we work on MT940 type messages, we focused our efforts on that.

The message itself consists of the following blocks:

{1:} Basic header block.

{2:} Application header block.

{3:} Sser header block.

{4:} Text block.

{5:} Trailer block.

Each of these blocks has a specific format to follow which I included the detailed tables in figure B in the appendix.

2. Provide Core API

Next, we looked for potential tools to use, since the customer wanted to replace their existing, third-party solution, we looked only into open-source APIs to use. The most complete and comprehensive open-source tool we found was the Provide core API.

Provide Core is an open-source Java library that can manage, model, parse and convert legacy FIN MT files with accuracy and security. It is a great choice for the company as they wanted to

replace the third-party solution with internal resources, and this tool will allow us to achieve that with extreme efficiency.

These are the core functionalities that we used from the Provide Core API [6]:

- Parser from any MT file to Java. Using the abstract class AbstractMT, we can parse any MT file and determine the message type. Subsequently, using the message type we can create the specific MTXXX object for the correct message type using type casting in Java. Using the newly created MTXXX object, we can read and handle each specific field for the different types.
- Builder from Java back to MT, as this will allow the company to construct the SWIFT MT messages to send to their customers. This part has not been implemented yet as it requires the real-life datasets and use cases the company has yet to provide us but promised to provide soon.
- Converters to both JSON and XML. In the initial development and testing part, we converted the files to JSON for convenience, if the company specifies otherwise, we can easily provide both options.

After opening and reading the string contents of a FIN file, we create an AbstractMT object and run the parse function on the string [6]:

```
AbstractMT msg = AbstractMT.parse(fin);
```

then we check the message type using AbstractMT.isType() function, and we create a new MT940 object like the following:

```
if(msg.isType(940)){  
    MT940 mt = (MT940) msg;  
}
```

After that we can use the parsed object to access each individual field as specified in the individual specifications for each message type and extract information as necessary. We can also use the information to create a new SWIFT MT message.

3. Build tool and database: Maven and MongoDB

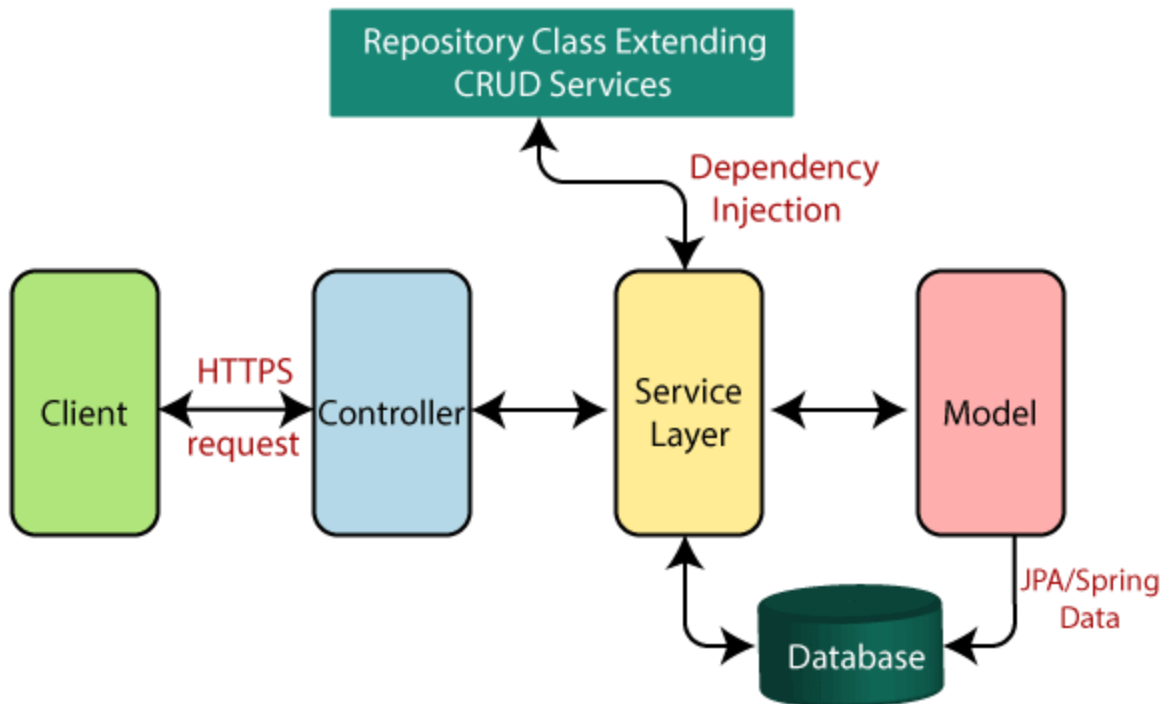
Next, we created a simple Java console application to test the API. We ran into issues with importing the JAR files necessary, so we investigated an efficient build tool to manage our dependencies. Our choice was between two popular build tools for Java, Maven and Gradle. We chose Maven for having an easier build process outside of the source code, as well as being more industry-adopted and the most popular Java build automation tool, while also having an easier learning curve. However, this comes at the expense of better performance by Gradle [7].

After setting up the dependencies and building the first part of our project, we have developed a basic MT fin to JSON converter and converted a sample MT940 file we found online in the Provide core test cases on GitHub. The successful conversion will create the JSON file with the message's unique reference number as its filename. In order to be able to store those JSON output files, we created a non-relational based database by using MongoDB Atlas. MongoDB is a NoSQL database and by using MongoDB Atlas, we can reach a server in cloud, which holds our data, from our local environment. MongoDB Atlas is a multi-cloud database service that simplifies deploying and managing databases. It provides us the opportunity to manage free clusters in a serverless instance or define a dedicated cluster configuration specifically for our application [8]. It is suggested to deploy the database to the same cloud provider and region as the applications to reduce latency and more standardized security. In our case, we use Amazon Web Services in the region of Paris.

4. Spring MVC

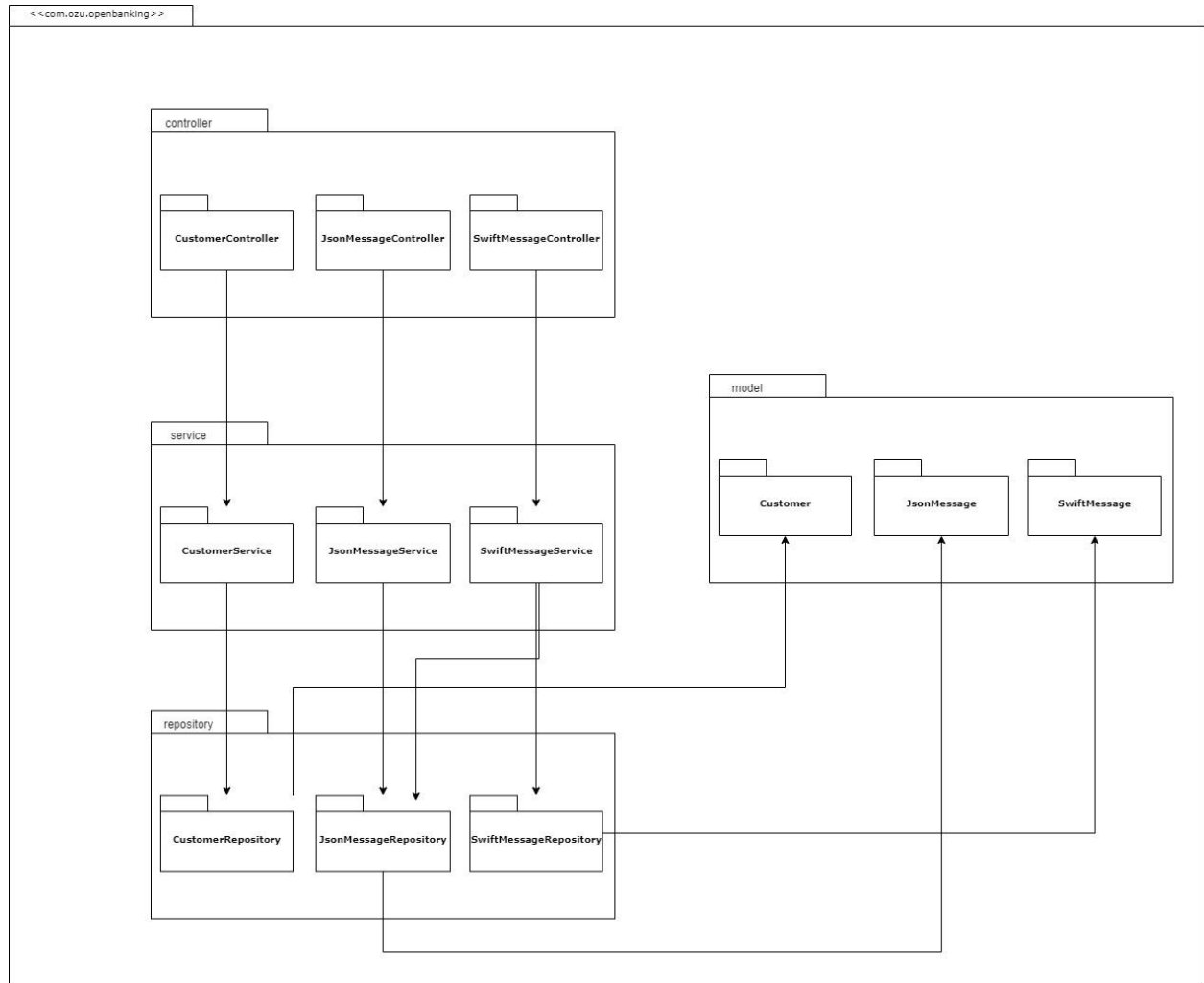
Next, for developing the web application, we researched the Spring MVC framework to understand its architecture, design patterns, and components for building web applications. By understanding how it simplifies the development process by providing features such as inversion of control (IoC), dependency injection (DI), and declarative programming. Additionally, we utilized Spring-Boot, which is built on top of the Spring framework, and it is a popular JAVA-based framework which simplifies the development of modular, production-grade applications. It does so by providing a streamlined approach to creating Java applications by simplifying configurations and reducing boilerplate code. Spring-boot includes an embedded web server to eliminate the need for external server deployment. It also provides pre-defined configurations allowing us to focus more on the application logic. Utilizing Spring-Boot, we created a web application to convert our console Java application into a RESTful API. Our Spring Boot application consists of a controller layer which serves the clients HTTP request. The controller then interacts with the service layer which contains the business logic and processes the request to modify the model and persists to the database (MongoDB) using the JPA repository through dependency injection. The following diagram describes the architecture flow of our Spring Boot application.

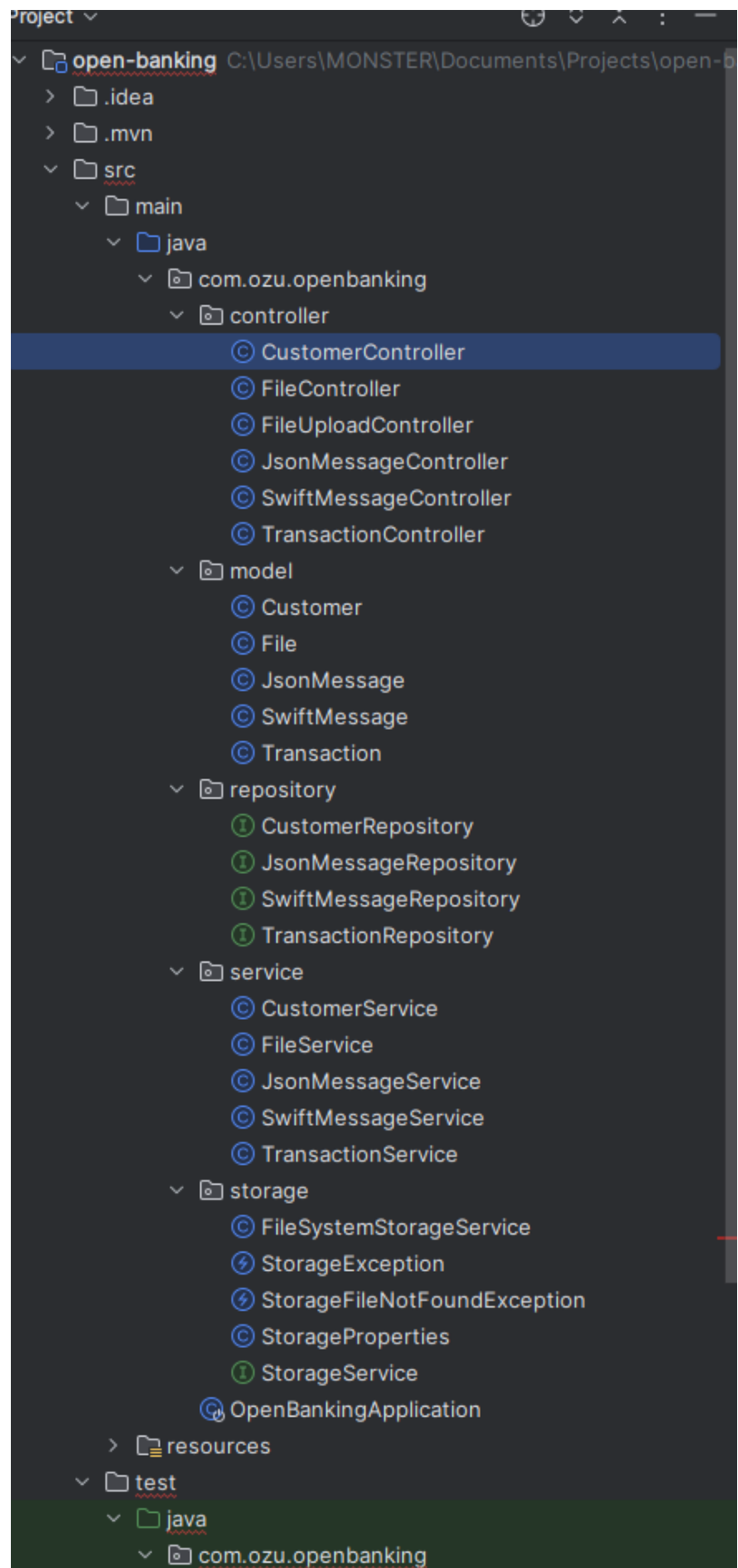
Spring Boot flow architecture



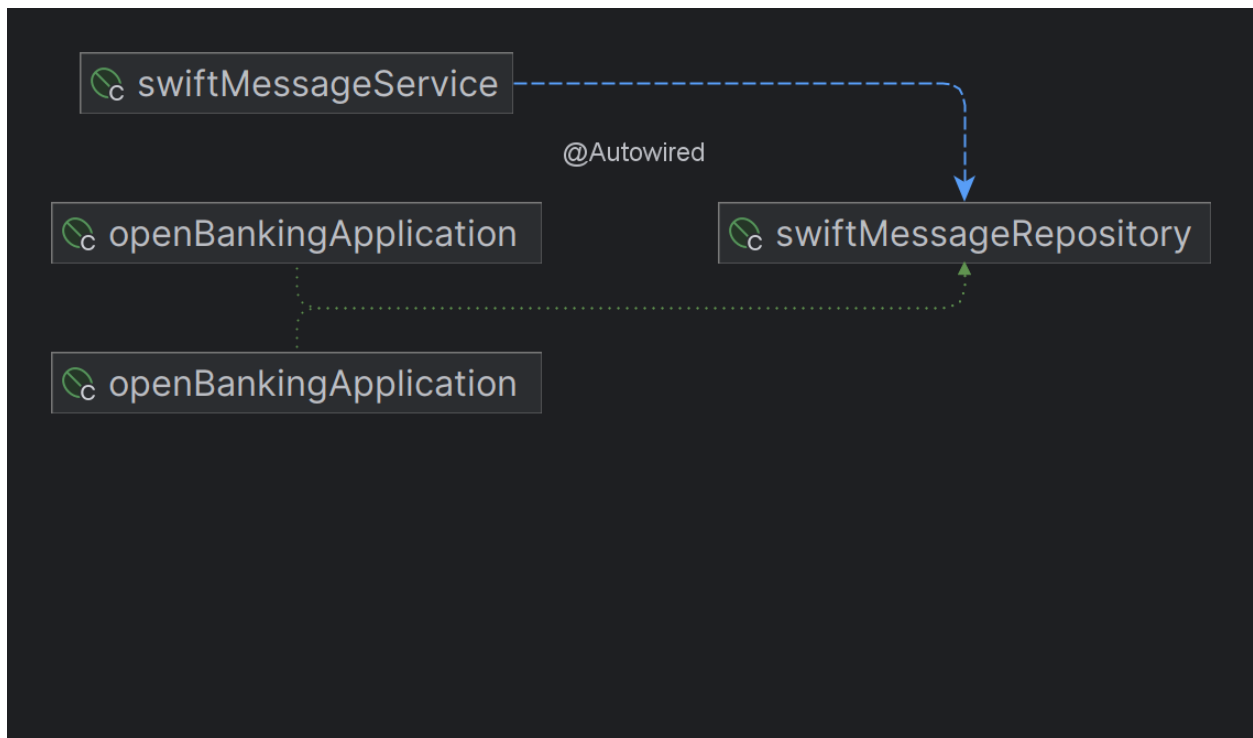
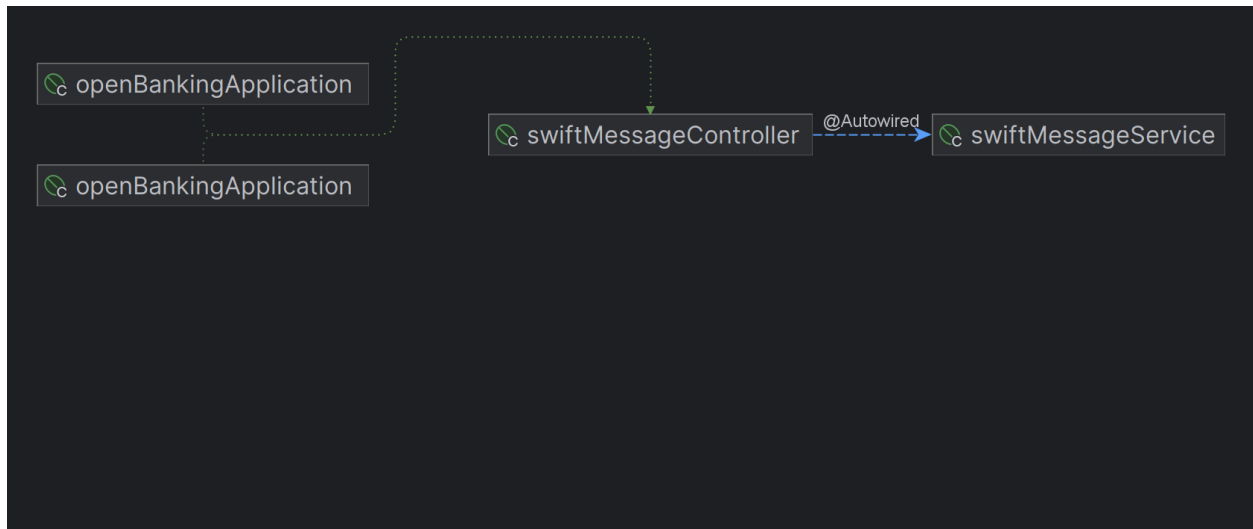
[9]

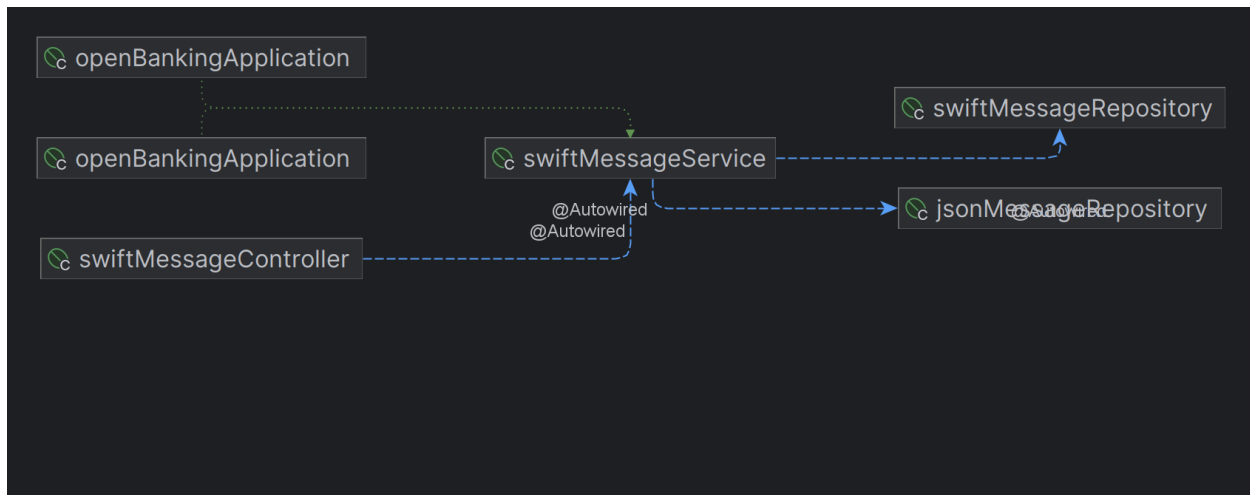
The following diagrams describe the general architecture of our program.





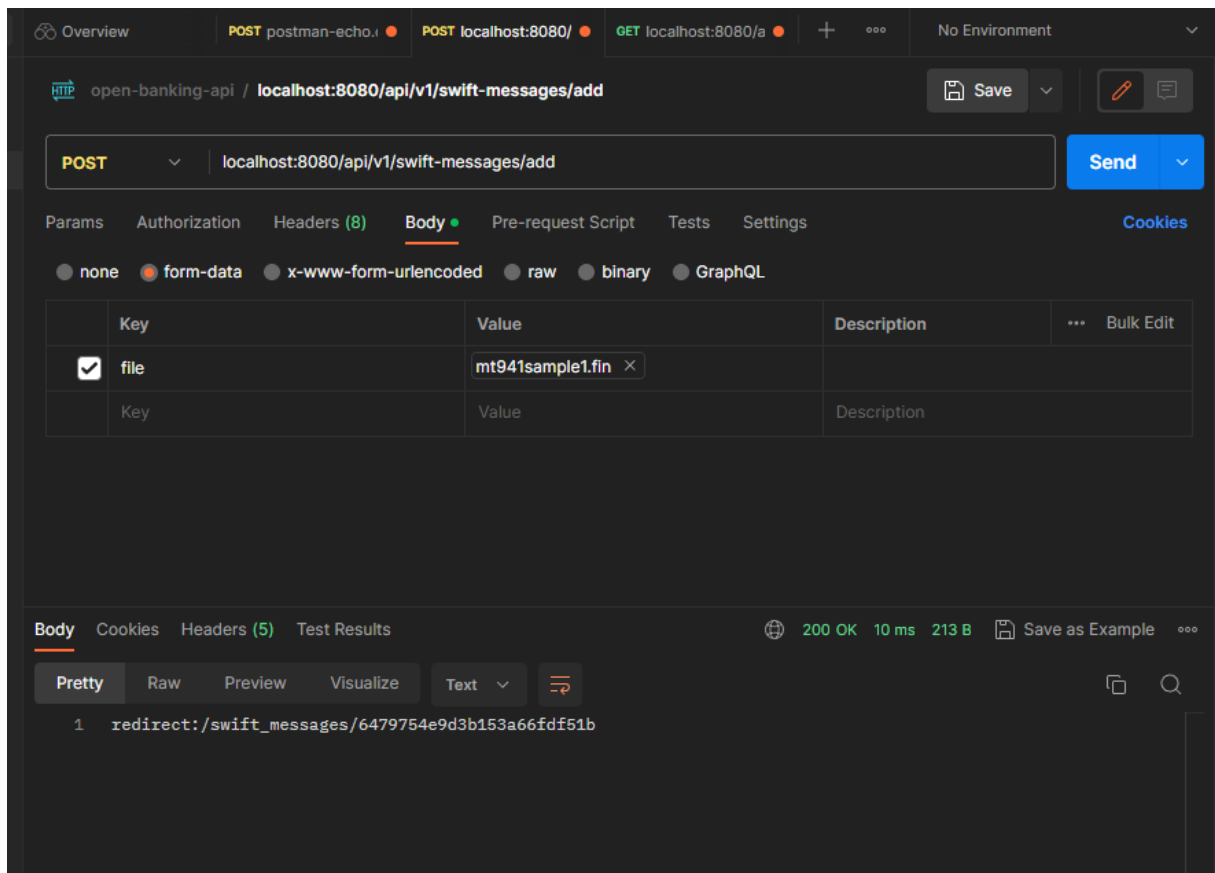
The following diagrams describe the Spring dependency diagrams for a SwiftMessage type. More in Appendix Q.





5. RESTful services

Also known as RESTful APIs (Application Programming Interfaces) are the set of guidelines and principles for implementing web services. They are built upon some key principles such as: Statelessness, resource-based, uniform interface representation-oriented. REST APIs are widely used for building web services that enable systems to interact over the HTTP protocol in a standardized and scalable manner. REST APIs use the standard HTTP methods (GET, POST, PUT, PATCH, DELETE) and expose resources accessible through a URI (Uniform resource identifier). It follows a uniform interface which uses a set of conventions for request and response status codes and formats and are stateless as in it does not need to store client-specific information between multiple requests, each request is standalone and must contain all the necessary data for the server to process it. In our application we used the controller layer to process HTTP requests and communicate with the service layer to execute the business logic and return the correct data and HTTP status codes. The complete implementation of the web application can be found in Appendix H-P. The following figure shows the POST command for uploading and converting a Swift MT fin message file using Postman to write the HTTP command.



6. Banking APIs

Since we reached the second phase of our project, we have searched for banking APIs to test their functionality to understand their working methods and spot the ones we can use their functionality by migrating then into our project. We had aimed to have a functionality as making money transfer between our accounts in the first phase. For those reasons, we decided to visit the API portals of Fibabanka and Akbank to test their APIs. A specific client key was required for authentication process, so we created accounts and send request for OAuth, which is the industry-standard for authorization, for both API portals [10]. While testing the REST APIs of the banks, we used both cURL (client URL) tool, which is fundamentally used to connect to the server by specifying the location and the data that wanted to be sent, to apply a HTTP method such as GET which is used to retrieve a resource from the server [11].

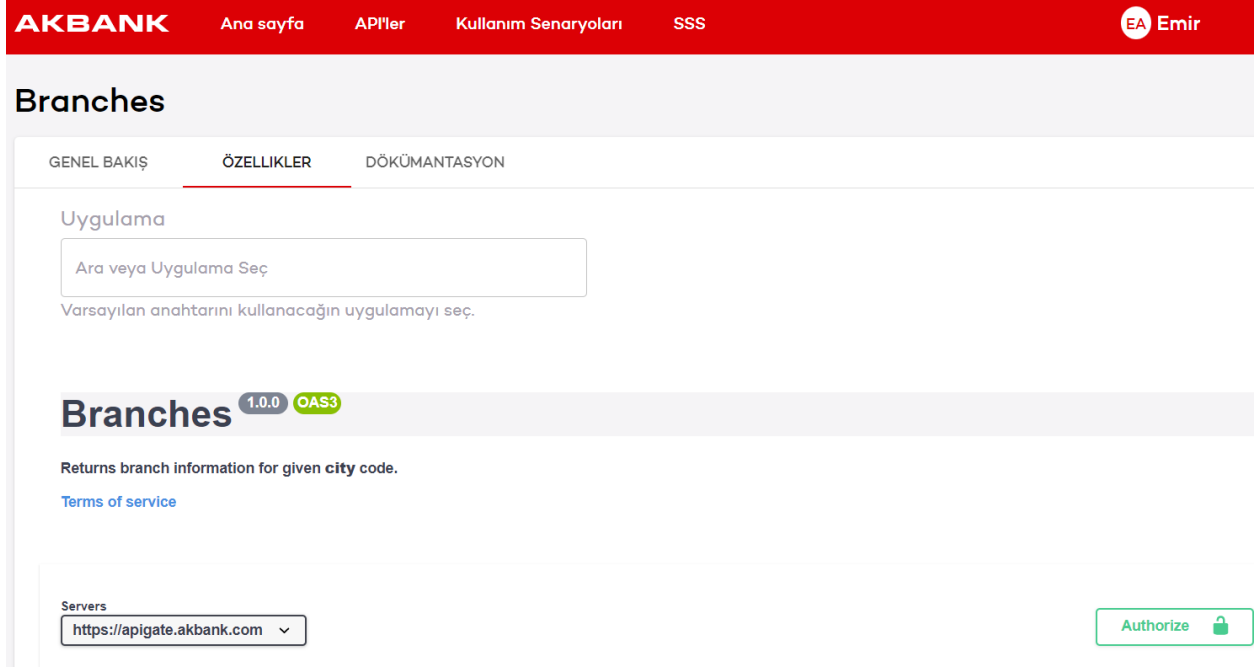


Figure-1: API Portal and the 'Branches' API of Akbank

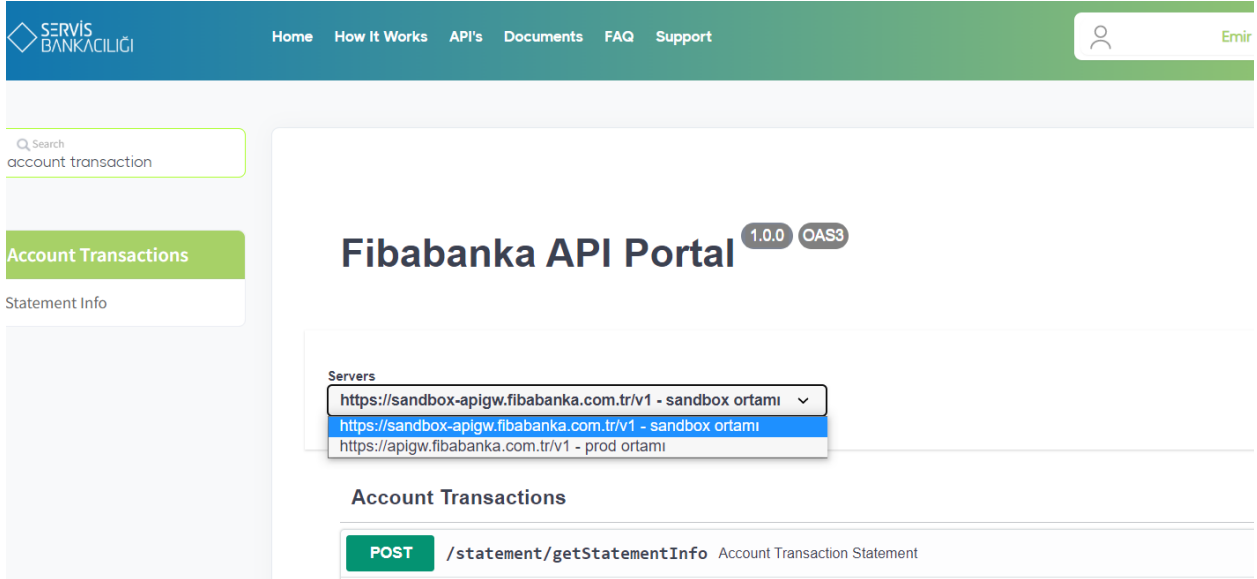


Figure-2: API Portal and 'Statement Info' API of Fibabanka

Some Examples of tested Banking APIs:

- In the API Portal of Akbank, 'Atms' which returns Akbank ATM locations with the given coordinates, was tested by using cURL tool in command line to retrieve requested information as shown below.

```
curl -X GET
"https://apigate.akbank.com/api/v1/general/atms/mock?latitude=41.008238&longitu
de=28.978359&radius=1000" -H "accept: application/json" -H "Authorization:
Basic ZW1pc19heWRuOkVtcmkzNWZiLQ==" -H "apikey:
170ff6bb3e49b949cc88ec64285f362087" -H "keySecret:
bec3e34cefc84becbcd3fa609a828607"
```

The response was provided in JSON format with success, and it is shown in Appendix[E] section.

- In Fibabanka API Portal, the 'StateInformation' API monitors available accounts with balances and get detail of transactions within different periods. The HTTP POST method is used by using cURL tool as shown below.

```
curl -X 'POST' https://sandbox-
apigw.fibabanka.com.tr/v1/statement/getStatementInfo' -H 'accept:
application/json' -H 'Content-Type: application/json' -d
'{"corporationCode": "31205", "accountNo": "123670517", "startDate":
"20210323", "endDate": "20210324", "summary": ""}'
```

The expected response in JSON format is shown in Appendix[F] section.

- The 'CreditInterestRates' API returns the current interest rates for consumer loans, mortgages and car loans. It was tested by sending request to server by using cURL tool as shown below.

```
curl -X GET
"https://apigate.akbank.com/api/v1/loan/retail/interestrates/mock?type=I" -H
"accept: application/json" -H "Authorization: Basic
ZW1pc19heWRuOkVtcmkzNWZiLQ==" -H "apikey: 170ff6bb3e49b949cc88ec64285f362087" -
H "keySecret: bec3e34cefc84becbcd3fa609a828607"
```

The response was provided in JSON format with success, and it is shown in Appendix[G] section [10].

Knowledge and skill set:

In this project, we used the knowledge and experience we accumulated from our computer science curriculum to design, research possible solutions and implement the project. We used the knowledge gained in the following courses:

Introduction to programming: We used Java as our programming language, which we learnt from this course initially, as well as the basics of programming.

Object oriented programming and software design patterns: We used object-oriented programming in designing and implementing our modular Spring MVC open banking application. Additionally using the Spring framework, itself which implements various design patterns such as Singleton, Prototype, Factory, and Template design patterns. For example, the Factory design pattern is a fundamental part of the Spring framework, the bean initialization in BeanFactory interface is based on it as well as the ApplicationContext, which is an extension of BeanFactory. ApplicationContext adds extra functionality to an application that is configurable with different contexts.

database management systems: We created and managed both SQL (SQLite) and a NoSQL (MongoDB) databases to store all our files. We executed various queries using a JDBC driver to access a local SQLite file server initially. Although we later found MongoDB to be more suited to our needs and used it as our main database, both locally and on the cloud deployed through MongoDB Atlas on AWS.

Web development (Restful APIs). We learnt about Restful APIs, by which any bank provides a PSD2 compliant interface through its Restful, banking APIs and requested authorization from various banks to use these APIs. Additionally, we converted our initial Java console application to a RESTful web service using Spring framework for the backend. We also used React.js to create an initial prototype for the frontend of our banking web application.

Engineering standards:

SWIFT MT messages adhere to the ISO 15022 specification. It is the old standard used in financial information, for exchange of electronic messages between financial participants. It was followed by the ISO 20022 specification which is the new global standard for financial information [12]. Under ISO 20022 swift message types will undergo some changes where the old MT format will be gradually replaced with the MX format for ISO 20022. This new message standard is designed to provide a single, standardized approach across financial services [13].

Payment Service Providers Directive (PSD2) is a European standard and regulation for electronic payment and financial services. Its purpose is to make payments more secure while boosting innovation and accelerating the adaptation of new technologies. PSD2 includes two main types of services, Payment initiation services (PIS); and Account Information Services (AIS). PIS allows providers to harmonize the functionality of providing online payment services. AIS enables the storage and collection of information from a customer's different bank accounts in

one place. Although not explicitly included in PSD2, APIs are the technical means which allow banks to comply with PSD2 regulations [14].

V. Results and Discussion

In the first part of our project, we completed an initial version of the Swift MT to JSON converter and performed some time measurements, the time complexity of the program is $O(N)$ when reading each file. The simple program takes any Swift MT file (in the test we used the FIN file format) and successfully parses the contents. Then the program converts the text data to JSON format using the Provide core API, the newly created JSON file has the reference number that was extracted from the SWIFT MT file as its filename. The following is a screenshot of the program, time measurements and the output JSON files. In the appendix we specified the complete details of the program and the output of the JSON file.

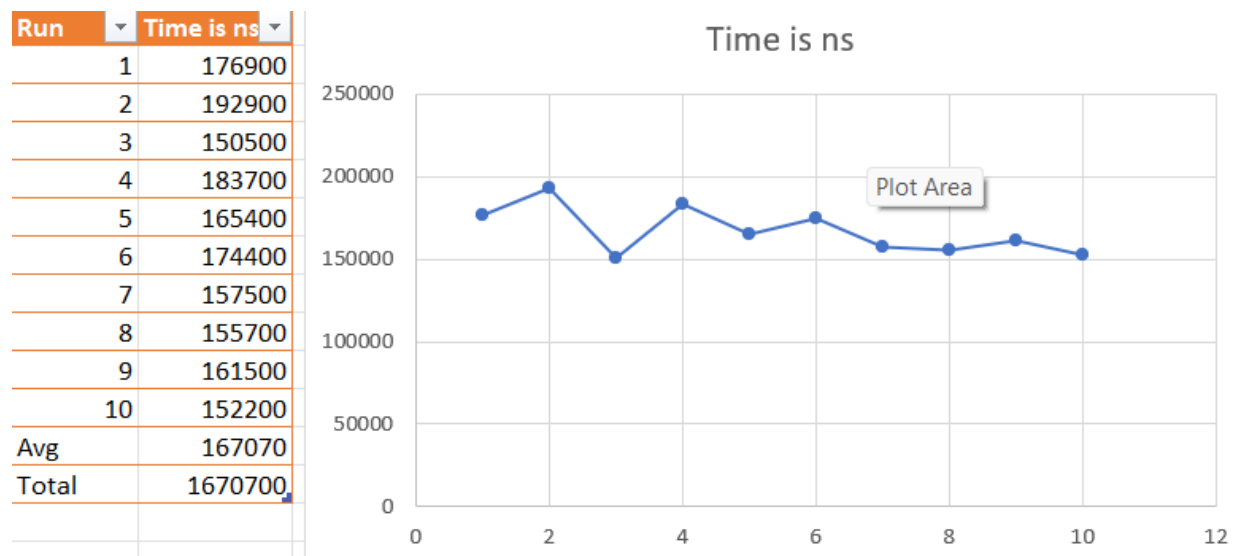


Figure A, time measurements of the parsing function in nanosecond.

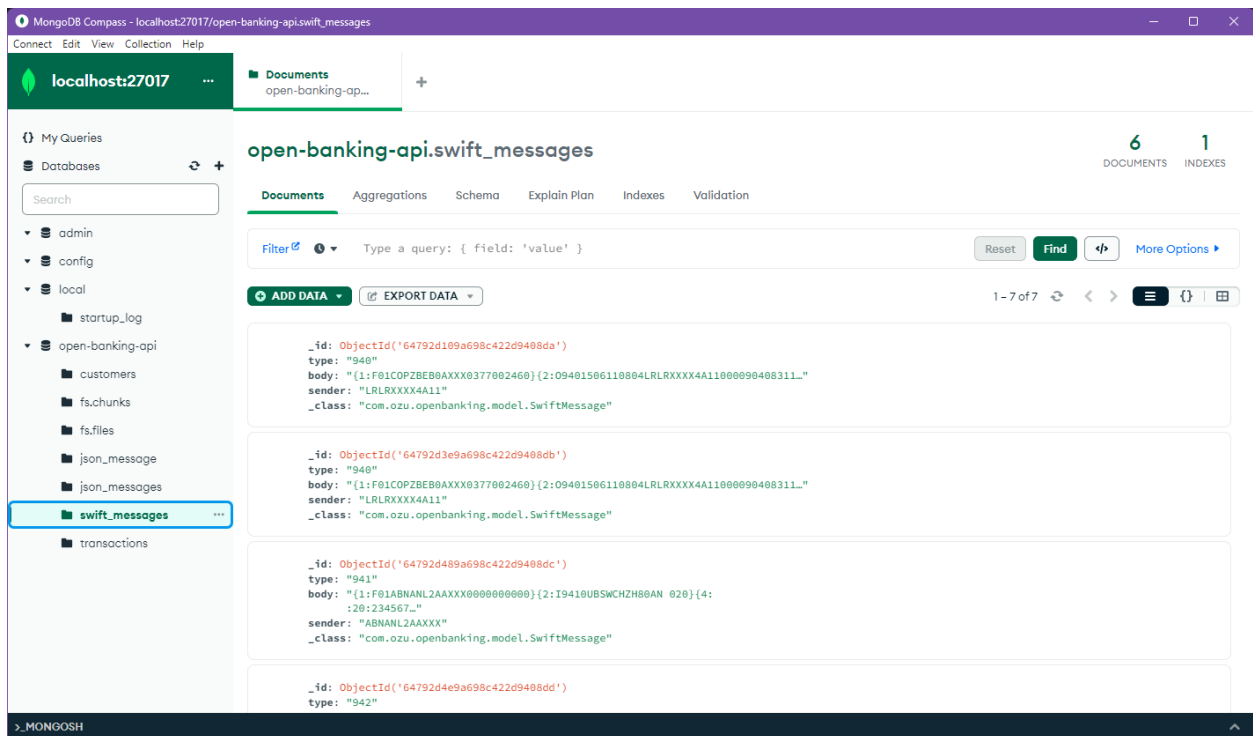
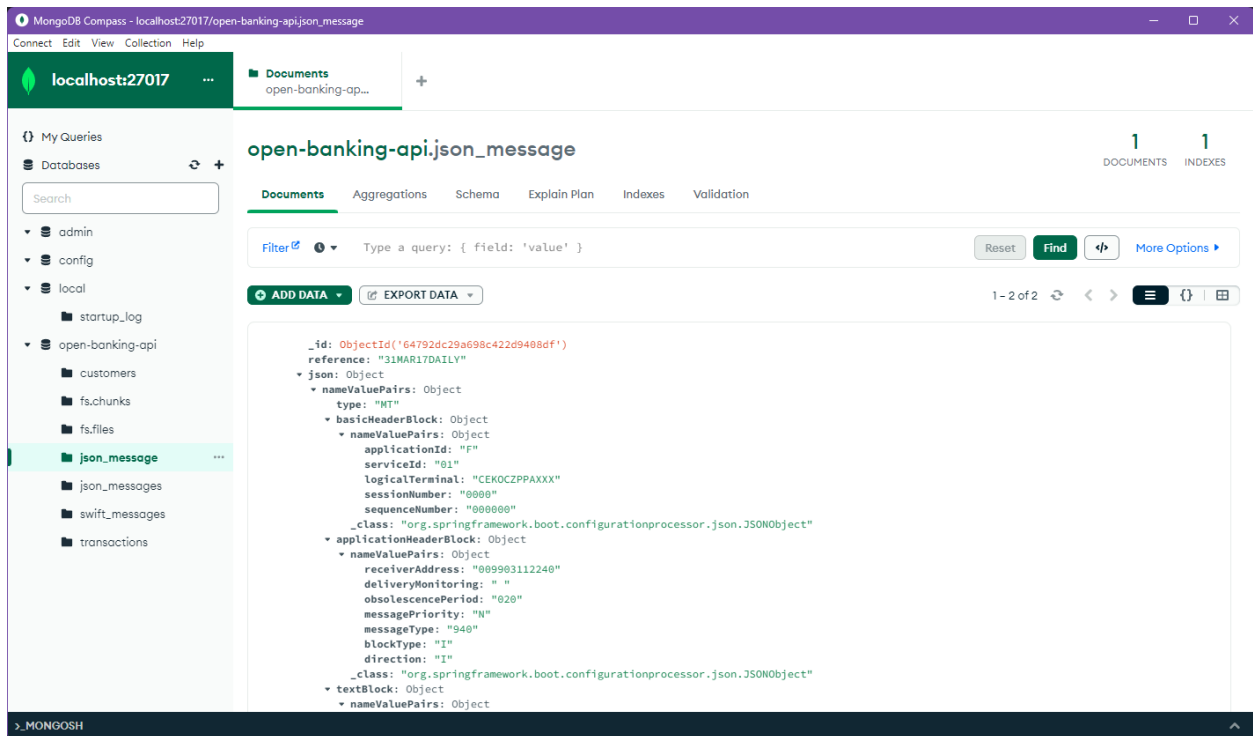
```

1  {} 31MAR17DAILY.json X
C: > Users > MONSTER > Documents > Projects > MTtoJSON2 > {} 31MAR17DAILY.json > ...
2  {
3    "type": "MT",
4    "basicHeaderBlock": {
5      "applicationId": "F",
6      "serviceId": "01",
7      "logicalTerminal": "CEKOCZPPAXXX",
8      "sessionNumber": "0000",
9      "sequenceNumber": "000000"
10   },
11   "applicationHeaderBlock": {
12     "receiverAddress": "009903112240",
13     "deliveryMonitoring": "",
14     "obsolescencePeriod": "020",
15     "messagePriority": "N",
16     "messageType": "940",
17     "blockType": "I",
18     "direction": "I"
19   },
20   "textBlock": {
21     "fields": [
22       {
23         "name": "20",
24         "reference": "31MAR17DAILY"
25       },
26       {
27         "name": "25",
28         "account": "0000000123456"
29       },
30       {
31         "name": "28C",
32         "statementNumber": "00065",
33         "sequenceNumber": "1"
34       },
35       {
36         "name": "60F",
37         "dCMARK": "C",

```

Figure B, JSON output

In the second part of the project, we worked on converting the simple Java console application into an open banking web application. We did that by using Spring framework to write the backend of our application, after which we used Postman to test the API endpoints for our backend. Finally, we used MongoDB compass as our GUI to show the results and to view the results in the database. The following is the MongoDB compass output of our web application.



Additionally, after creating accounts in the API Portals of Fibabanka and Akbank. We tested the REST APIs in those portals and results of the testing process are shown at the table below.

Bank Name ▼	API Name ▼	Requires Oauth ? ▼	Response ▼
AKBANK	Retail Customer Loan	Yes	interval server error
AKBANK	Atms	Yes	SUCCESS
AKBANK	Branches	Yes	Unauthorized valid key
AKBANK	Credit Interest Rates	Yes	SUCCESS
AKBANK	Consumer Loan Calculation	Yes	interval server error
AKBANK	Cities	Yes	interval server error
FIBABANKA	Customer Credit Cards	No	network failure
FIBABANKA	Customer Debit Cards	No	network failure
FIBABANKA	Query Lending	No	network failure
FIBABANKA	Statement Info	No	network failure

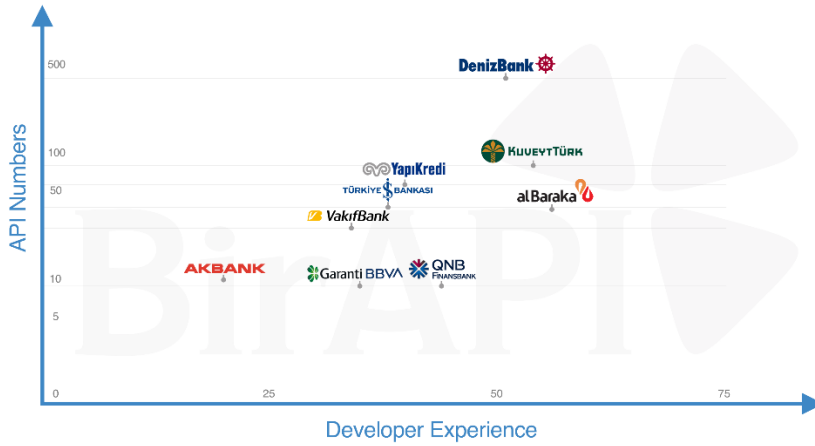
Figure C, results of the API testing

The tested APIs are mentioned in Figure C and the given responses by the server for each API are described in the ‘Response’ column. ‘Atms’ and ‘Credit Interest Rates’ were APIs that worked and returned their expected responses in JSON format which is shown in the Appendix[E] and Appendix[G] sections. Apart from those two APIs, others, that were tested in the portals and shown on table, returned several error messages due to server errors, network failures or invalid keys at authentication. Authorizing the key is the primary requirement for accessing the server for the usage of the APIs. Although some APIs in Fibabanka API Portal didn’t require authorized valid keys, which is described in ‘Requires Oauth’ column, they return a respond as network failure. As we reported the issue and contacted Fibabanka, valid keys to access the testing environment are defined for our IP addresses.

VI. Related Work

The reason why the company requested this tool is because of their reliance on proprietary third-party software for this purpose. As we researched the possible solutions, we came across several simple implementations on GitHub, and the Provide Core API. There were no options to consider using as more comprehensive and efficient as the Provide Core API as that is what we chose to use in our program.

Moving forward, we wanted to integrate open banking APIs to add more functionality to our tools, so we have researched and compared different Banking APIs from banks across Turkey.



- API numbers axis shows the number of open APIs that banks have published on their portals.
- Developer experience score is based on the survey results of active users about API documentation, data functionality, accessibility, etc. (Evaluated out 100)



www.birapi.com

[15]

As we researched the different banking APIs across the following Turkish banks: Ziraat Bank, Is Bank, QNB finansbank, Garanti bank, Deniz bank and YapıKredi Bank. I found that most of those banks only have APIs for basic functions such as checking customer and transaction information, as well as finding the nearest bank and ATM machines. The most notable collection of banking APIs belonged to the following:

- Garanti bank has a few more useful APIs, such as a P2P money transfer API using phone numbers, called "Cepbank Send Money". There is also another potentially useful API, the "Create QR service" [16].
- Is Bank has a comprehensive suite of APIs, these are some of their useful APIs: Campaigns, HGS Balance, Payment Plans, Repayment Schedule, Loans, Credit Scores, Customer Accounts, exchange rates, bill payments and money transfers [17].
- YapıKredi Bank had the most comprehensive API documentation with features such as Bill payment, EFT/Remittance to customer card/name/account, exchange money, currency rates, open and close accounts etc [18].

- Deniz Bank has the biggest collection of APIs, with categories such as Account activities, Money transfers, E-commerce site credit, investments, stocks, loans, credits, invoice, tax and many more [19].

VII. Conclusion and Future Work

As we conclude the current progress of our project, we have implemented an initial Java console application version of our SWIFT MT to JSON file converter. This simple program takes the SWIFT MT940 and parses it into contents and then converts it to JSON file with time measurement. Afterwards we experimented with different Database Management Systems (DBS) including both SQL and NoSQL databases, as well as local or cloud database servers. We expanded our program to include more message types and we generated more legacy Swift Message files to test our program on different Swift MT message types including MT 940, MT 941 and MT 942. Afterwards we created a new web application to convert our Java application to a RESTful web service using Spring framework in the backend and MongoDB Compass, a database viewer for the frontend. We used POSTMAN to test the API endpoints of our web application. Additionally, we contacted different banks across Turkey to gain access to open banking APIs and were able to receive approval and test some of their APIs.

The Impacts of the project:

Considering the functional requirements of the company, they will be able to successfully make transactions with their customers in countries that use the legacy Swift MT messages without depending on proprietary banking APIs. By using our open banking software solution, the company will be able to do their cash management while having information about the status of their customers using their own API. By following PSD2 requirements, the API we developed will be standardized to be used in the current industrial environment and it becomes an industrial solution.

Possible Ethical Issues:

As we have mentioned, the customer data of the company is a critical need for us to test our project. Therefore, the company has met some of their customers to get permission to use their financial information for our Fintech project and they decided to send petitions to banks demanding their data. The security of the data is an important concept of our project because an open banking ecosystem may include various players such as TPPs, data providers, customers, regulators and government agencies so there are a lot of potential points of failure when the share of data increases. By following PSD2 requirements, the violation, manipulation and theft of the data will be avoided by multi-factor authentication (MFA).

Our Future Work:

In the future, after gaining the required authorizations from banks, we plan on creating the frontend of our open banking web application. We plan to use the frontend to better utilize our swift MT converter as a web service on our website. Additionally, we plan to utilize the authorized APIs to add some extra functionalities to our web application such as reading account data across multiple banks and successfully completing simple financial transactions. Meanwhile, we plan to expand our database with more Swift message types and to do accurate mapping of Swift MT to Swift MX messages based on the requirements of clients in the financial technology sector. Finally, we would like to create rigorous test cases for type checking of our data as well as HTTP request.

Acknowledgements

We would like to express our sincerest gratitude to Prof. Ismail Ari, who has guided us every step of the way and provided us with invaluable information to better understand the overall context of our project. Even in the face of major delays in our project he encouraged us to give it our all and to continue working and to never give up. He provided us with any information we needed plus more and gave us a glimpse into his vast and impeccable expertise.

We would also like to express our thanks to Movetech and Mr. Ahmet Enes Ograğ for giving us the opportunity to work on an industry project and providing us with the information and data we need to complete our project.

Finally, we would like to thank Prof. Furkan Kırac for providing us with all the information and templates we needed to start and work on our project.

References

- [1] SWIFT, "MySWIFT knowledge Centre," [Online]. Available: <https://www2.swift.com/knowledgecentre/products/Standards%20MT/publications>. [Accessed 6 January 2023].
- [2] IR, "What Businesses Need To Know About The SWIFT Messaging Format," IR, [Online]. Available: <https://www.ir.com/guides/swift-message-format>. [Accessed 6 January 2023].
- [3] D. Harrington, "What is PSD2 Compliance and What Does it Mean for Your Business?," 26 July 2021. [Online]. Available: <https://www.varonis.com/blog/psd2>.
- [4] R. Chejkova-Nikolov, "Interoperability of bank statements: A case study," in *2015 38th International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)*, Opatija, 2015.
- [5] A. Madhavi and B. Sreedivya, "A Novel Bank Statements Reconciliation Using Message Transfer940 Parser," in *2017 IEEE International Conference on Computational Intelligence and Computing Research (ICCIC)*, Coimbatore, 2017.
- [6] Provide International S.A., "Provide Core Overview," [Online]. Available: <https://dev.providesoftware.com/SRU2022/open-source/core/#learn-by-reading>. [Accessed 6 January 2023].
- [7] T. Gregory, "Maven vs. Gradle in-depth comparison," [Online]. Available: <https://tomgregory.com/maven-vs-gradle-comparison/>. [Accessed 6 January 2022].
- [8] MongoDB, "MongoDB Atlas. Fully managed MongoDB in the cloud.," [Online]. Available: https://www.mongodb.com/cloud/atlas/lp/try4?utm_source=google&utm_campaign=search_gs_pl_evergreen_atlas_general_prosp-brand_gic-null_emea-tr_ps-all_desktop_eng_lead&utm_term=mongo%20data%20base&utm_medium=cpc_paid_search&utm_ad=e&utm_ad_campaign_id=203006. [Accessed 6 January 2023].
- [9] Java T Point, "Spring Boot Architecture," [Online]. Available: <https://www.javatpoint.com/spring-boot-architecture>. [Accessed 2 June 2023].
- [10] A. Parecki, "OAuth 2.0," [Online]. Available: <https://oauth.net/2/>. [Accessed 30 May 2023].
- [11] J. Saladas, "What is cURL and how does it relate to APIs?," IBM, 22 February 2021. [Online]. Available: <https://developer.ibm.com/articles/what-is-curl-command/>. [Accessed 02 June 2023].

- [12] IR, "What is ISO 20022 and How is it Changing?," [Online]. Available: <https://www.ir.com/guides/what-is-iso-20022>. [Accessed 6 January 2022].
- [13] Gresham Technologies plc, "SWIFT ISO 20022 migration: Everything you need to know about MT to MX conversion, mapping and more," Gresham Technologies plc, [Online]. Available: <https://www.greshamtech.com/iso-20022-migration-mt-mx-conversion>. [Accessed 26 2023].
- [14] Banco Bilbao Vizcaya Argentaria, S.A., "Everything you need to know about PSD2," 21 2 2023. [Online]. Available: <https://www.bbva.com/en/everything-need-know-psd2/>.
- [15] S. Eyidilli, "BirAPI gives clues about Turkish banks' Open Banking strategies," 30 November 2020. [Online]. Available: <https://www.challengerinsider.com/blog/birapi-gives-clues-about-turkish-banks-open-banking-strategies>.
- [16] Garanti Bank, "Public API," [Online]. Available: <https://developers.garantibbva.com.tr/pages/apiList.html>. [Accessed 6 January 2022].
- [17] Is Bank, "Bank APIs," [Online]. Available: <https://developer.isbank.com.tr/Pages/apis.aspx>. [Accessed 6 January 2022].
- [18] Yapi Kredi bank, "API Portal," [Online]. Available: <https://apiportal.yapikredi.com.tr/apiresources/explorer>. [Accessed 6 January 2022].
- [19] Deniz Bank, "APIs," [Online]. Available: <https://apiportal.denizbank.com/apimarket>. [Accessed 6 January 2022].
- [20] W3 Schools, "JSON - Introduction," [Online]. Available: https://www.w3schools.com/js/js_json_intro.asp. [Accessed 6 January 2023].
- [21] S. Daley, "What Is Fintech? Financial Technology Definition," 18 August 2022. [Online]. Available: <https://builtin.com/fintech>.
- [22] O. Klein, "Is OpenBanking and PSD2 replacing the SWIFT-network on the long run?," 3 December 2018. [Online]. Available: https://www.linkedin.com/pulse/openbanking-psd2-replacing-swift-network-long-run-dr-olaf-klein/?trk=related_article_Is%20OpenBanking%20and%20PSD2%20replacing%20the%20SWIFT-network%20on%20the%20long%20run%3F%20_article-card_title.

Appendix

A. SwiftMTtoJSON converter code.

```
package org.example;

import com.providesoftware.swift.model.SwiftMessage;
import com.providesoftware.swift.model.mt.AbstractMT;
import com.providesoftware.swift.model.mt.mt9xx.MT940;
import com.providesoftware.swift.model.mt.mt9xx.MT941;
import com.providesoftware.swift.model.mt.mt9xx.MT942;

import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;

public class App<T> {

    public static void main(String[] args) throws IOException {

        //Read file list

        Path filePath = Path.of("sample.fin");

        File directoryPath = new
File("C:\\Users\\MONSTER\\Documents\\Projects\\MTtoJSON2\\data");

        File filesList[] = directoryPath.listFiles();

        System.out.println("List of files and directories in the specified
directory:");

        //get the list of FIN files we want to convert to JSON

        for(File file : filesList) {

            System.out.println("File name: "+file.getName());

            System.out.println("File path: "+file.getAbsolutePath());

            System.out.println("Size :"+file.getTotalSpace());

            System.out.println(" ");

            filePath = Path.of(file.toURI());
```

```

        String sFin = Files.readString(filePath);

        //Check for service ID and if it is a system message, then gather the
actual MT from unparsed text

        SwiftMessage sm = SwiftMessage.parse(sFin);
        if (sm.isServiceMessage()) {
            sm = SwiftMessage.parse(sm.getUnparsedTexts().getAsFINString());
            sFin = sm.toString();
        }
        //System.out.println(sFin);
        AbstractMT mtMsg = AbstractMT.parse(sFin);

        String sender = mtMsg.getSender();
        String type = mtMsg.getMessageType();
        //AbstractMT msg = AbstractMT.parse(Lib.readResource("sample.txt", null));

        //Check the SWIFT MT Format specification and initialize the correct
object

        MTBuilder builder= new MTBuilder(mtMsg);

        if (mtMsg.isType(940)){
            MT940 mt = (MT940) mtMsg;

            //The reference number should be used as the identifier for the
database entry

            //this is how to access the reference number/Field 20
            System.out.println("Reference: " + mt.getField20().getValue());

            Database.insertToTable("MT940", mt.getField20().getValue(),
mt.getField25().getAccount(), mt.toJson());

            String output = mt.getField20().getValue() + ".json";
            FileWriter toFile = new FileWriter(output);
            toFile.write(mt.toJson());
            toFile.close();
        }
        else if(mtMsg.isType(941)){
            MT941 mt = (MT941) mtMsg;

```

```

        //The reference number should be used as the identifier for the
database entry

        //this is how to access the reference number/Field 20
        System.out.println("Reference: " + mt.getField20().getValue());

        Database.insertToTable("MT941", mt.getField20().getValue(),
mt.getField25().getAccount(), mt.toJson());

        String output = mt.getField20().getValue() + ".json";

        FileWriter toFile = new FileWriter(output);

        toFile.write(mt.toJson());

        toFile.close();

    }

    else if(mtMsg.isType(942)){

        MT942 mt = (MT942) mtMsg;

        //The reference number should be used as the identifier for the
database entry

        //this is how to access the reference number/Field 20
        System.out.println("Reference: " + mt.getField20().getValue());

        Database.insertToTable("MT942", mt.getField20().getValue(),
mt.getField25().getAccount(), mt.toJson());

        String output = mt.getField20().getValue() + ".json";

        FileWriter toFile = new FileWriter(output);

        toFile.write(mt.toJson());

        toFile.close();

    }

    else{

        System.out.println("Unsupported message type.");

    }

}

}

}

```

B. MT940 message specification. Retrieved from:

<https://www.csob.cz/portal/documents/10710/4165543/mt940-statements-en.pdf>

Description	Required field	Length and format
<CR><LF>:20: Statement date and frequency (date in the DDMMYY format, frequency DAILY, ...)	x	4 a P 16 a V
<CR><LF>:25: Account number in the ABO format, leading zeros and bank code can be provided Note: in case there is prefix in the account number, leading zeros are used (account 19-19/0300 is presented either as 0300/190000000019 or simply as 190000000019, the latter is valid for accounts held in IBIS system). For accounts without an ABO version, an IBIS version of account number is specified.	x	4 a P 35 a V
<CR><LF>:28C: Statement number and page number Statement number is five characters long (completed with leading zeros), a slash is followed by the page number Note: pagination is currently not used, the page number is always 1	x	4 a P 5 n P / 2 n V
<CR><LF>:60x: Balance; x will be replaced by: – F for opening balance (statement beginning) – M for subtotal (if the statement contains multiple pages) Note: statement pagination is not used (subtotal is never specified)	x	5 a P
Subfield 1: Balance sign, C for credit, D for debit	x	1 a P
Subfield 2: Balance date in the YYMMDD format	x	6 n P
Subfield 3: Balance currency (CZK for Czech koruna,...)	x	3 a P
Subfield 4: Balance (without leading zeros, decimal places separated by a comma)	x	15 n V
<CR><LF>:61: Account movement (transactions, repeats if multiple movements are recorded)	x	4 a P
Subfield 1: Foreign currency date in the YYMMDD format (typically identical to the posting date)	x	6 n P
Subfield 2: Posting date in the MMDD format	x	4 n P
Subfield 3: Type of posting – C for credit, D for debit, RC for credit cancellation, RD for debit cancellation (cancellation is used for corrective posting)	x	2 a V
Subfield 4: Currency type (not used)	-	1 a V
Subfield 5: Transaction amount, without leading zeros and with a decimal comma (if relevant)	x	15 n V
Subfield 6: Text key – FMSC for domestic movements, NMSC for other movements	x	4 a P
Subfield 7: Client's reference (if no reference is given, one space instead)	x	16 a V
Subfield 8: //Transaction number (bank reference), 16 characters preceded by two slashes	-	16 a V
<CR><LF>Subpole 9: Additional information for selected transaction types, preceded by a slash and containing: – for selected cross-border transactions or FX conversions: information about the original payment amount in the /OCMT/XXXNNN,NN format, where XXX is a currency ISO code and NNN,NN is an original amount in this currency; optionally also with FX rate used for conversion (xxx.yyyyyy) – for incoming domestic transactions from another bank: date of debiting from the counterparty's account in the YYMMDD format – position 29–34	-	34 a V

<CR><LF>:86: More information about the transaction – up to 6 lines separated by <CR><LF>. The contents of this field is described in the next section; standard length defined by the SWIFT format (6 x 65 characters) is not always kept	x	
<CR><LF>:62x: Balance; x will be replaced by: – F for closing balance (end of statement) – M for subtotal of a given statement page (should there be more pages) Note: statement pagination is not used (subtotal is never specified)	x	5 a P
Subfield 1: Balance sign, C for credit, D for debit	x	1 a P
Subfield 2: Balance date in the YYMMDD format	x	6 n P
Subfield 3: Balance currency (CZK for Czech koruna, ...)	x	3 a P
Subfield 4: Balance (without leading zeros, decimal places separated by a comma)	x	15 n V
<CR><LF>:} End of statement	x	2 a P

Transaction detail – field 86

Detailed information on specific transaction is available in :86 field. The usage of this field might be bank-specific, below description is valid for CSOB CZ account statements. For ČSOB accounts, individual attributes of a transaction are specified in subfields, the "}" sign usually separates subfields from each other. The transaction detail is different for domestic payments (in CZK), cross-border payments (in a foreign currency) and other movements (cash deposits, payment card transactions, fees, etc.).

Domestic payments (TPS)

Subfield	Contents	Length and format
111	Transaction type – 111 for domestic payments	3 n P
?00	Counterparty name	27 a V
?20	Counterparty's account (prefix-account/bank), leading zeros are used	6 n P-10 n P/4 n P
<CR><LF>?21	Variable code	VS:10 n P
?22	Specific code	SS:10 n P
?23	Constant code	KS:4 n P
<CR><LF>?24	Message to the beneficiary – part one	27 a V
?25	Message to the beneficiary – part two	27 a V
<CR><LF>?26	Message to the beneficiary – part three	27 a V
?27	Message to the beneficiary – part four	27 a V
<CR><LF>?28	Counterparty's variable code	VS:10 n P
?29	Counterparty's specific code	SS:10 n P

Notes:

- The Message to the beneficiary fields: if relevant content is missing, will be filled with single dot.
- The fields with codes: leading zeros can be used depending on how they have been specified in the payment.

Cross-border payments (ZPS)

Subfield	Contents	Length and format
030	Transaction type (030 for cross-border payments)	3 n P
?00	Kurs:xxxxx,yyyyy – the exchange rate used for the currency conversion, 6 decimal places (Kurs means exchange rate)	27 a V
?20	Counterparty name	27 a V

721	Transaction type (ZAHRAICNI PLATBA = CROSS-BORDER PAYMENT or SCT = for SEPA credit transfer)	27 a V
<CR><LF>722	Purpose of payment – part one	27 a V
723	Purpose of payment – part two	27 a V
<CR><LF>724	Purpose of payment – part three	27 a V
725	Purpose of payment – part four	27 a V
<CR><LF>726	Purpose of payment – part five	27 a V
727	POPL ZAHRA:XXXXXXXXXXXX,NN – fee paid to a another bank, where XXX is a currency code and NNN a fee amount	27 a V
730	Counterparty's bank code (typically SWIFT code)	11 a V
<CR><LF>731	Counterparty's account number	32 a V
732	Counterparty name – part one	27 a V
<CR><LF>733	Counterparty name – part two	27 a V
//CHGS/	Fees charged by the bank with which the account is held (i.e. ČSOB) in the XXXXXXXXXXXX,NN format, where XXX is a currency code and NNN a fee amount	14 a P

Notes:

- The Purpose of payment fields: if relevant content is missing, will be filled with dot.
- If information about fees is unknown at the moment when a statement is being generated, the information is not specified.

Other movements

Subfield	Contents	Length and format
040	Transaction type – 040 for other payments	3 n P
700	Text description of the transaction	27 a V
720	Variable code	VS:10 n P
721	Message to the beneficiary / transaction detail – part one	27 a V
<CR><LF>722	Message to the beneficiary / transaction detail – part two	27 a V
723	Message to the beneficiary / transaction detail – part three	27 a V
<CR><LF>724	Message to the beneficiary / transaction detail – part four	27 a V
725	Specific code	SS:10 n P
726	Constant code	KS:4 n P

Notes:

- Fields are filled differently depending on the type of transaction (fees, cash pooling, cash transactions, card payments, transfers between one's own accounts, etc.).
- The variable and specific code are used e.g. for cash deposits at a ČSOB ATM or for card transactions in general.
- The Message to the beneficiary fields: if relevant content is missing, will be filled with dots.

C. JSON output from the conversion.

```
{
  "type": "MT",
  "basicHeaderBlock": {
    "applicationId": "F",
    "serviceId": "01",
    "logicalTerminal": "CEKOCZPPAXXX",
    "sessionNumber": "0000",
    "sequenceNumber": "000000"
  },
  "applicationHeaderBlock": {
    "receiverAddress": "009903112240",
    "deliveryMonitoring": " ",
    "obsolescencePeriod": "020",
    "messagePriority": "N",
    "messageType": "940",
    "blockType": "I",
    "direction": "I"
  },
  "textBlock": {
    "fields": [
      {
        "name": "20",
        "reference": "31MAR17DAILY"
      },
      {
        "name": "25",
        "account": "0000000123456"
      },
      {
        "name": "28C",
        "statementNumber": "00065",
        "sequenceNumber": "1"
      }
    ]
  }
}
```

```
},
{
  "name": "60F",
  "dCMark": "C",
  "date": "170330",
  "currency": "CZK",
  "amount": "100,00"
},
{
  "name": "61",
  "valueDate": "170331",
  "entryDate": "0331",
  "debitCreditMark": "D",
  "amount": "1,20",
  "transactionType": "N",
  "identificationCode": "MSC",
  "referenceForTheAccountOwner": "12345678909876",
  "referenceOfTheAccountServicingInstitution": "3150636703",
  "supplementaryDetails": "/OCMT/CZK1,20"
},
{
  "name": "86",
  "narrative": "030?00Kurs:1,000000?20NAZEV PROTISTRANY?21ZAHNANICNI PLATBA",
  "narrative2": "?22testovaci prevod ZPS?23.",
  "narrative3": "?24.?25.",
  "narrative4": "?26.?27POPL.ZAHR:CZK0,00?30CEKOCZPP",
  "narrative5": "?31CZ6303000000000000654321?32NAZEV PROTISTRANY",
  "narrative6": "?33ADRESA PROTISTRANY"
},
{
  "name": "61",
  "valueDate": "170331",
  "entryDate": "0331",
  "debitCreditMark": "D",
```

```
"amount": "1,10",
"transactionType": "F",
"identificationCode": "MSC",
"referenceForTheAccountOwner": " ",
"referenceOfTheAccountServicingInstitution": "1720170331000001"
},
{
  "name": "86",
  "narrative": "111?00NAZEV PROTISTRANY?20000000-0000654321/0300",
  "narrative2": "?21VS:7987613246?22SS:8976343437?23KS:0123",
  "narrative3": "?24testovaci prevod TPS?25.",
  "narrative4": "?26.?27.",
  "narrative5": "?28VS:7987613246?29SS:8976343437"
},
{
  "name": "61",
  "valueDate": "170331",
  "entryDate": "0331",
  "debitCreditMark": "C",
  "amount": "2,30",
  "transactionType": "N",
  "identificationCode": "MSC",
  "referenceForTheAccountOwner": " ",
  "referenceOfTheAccountServicingInstitution": "501509291000"
},
{
  "name": "86",
  "narrative": "040?00Vklad hotovost ATM 1111?20VS:0000123456?21Vklad hotovost
ATM 1111",
  "narrative2": "?22CSOB Radlicka?23test vklad ATM",
  "narrative3": "?24.?25SS:0012345678?26KS:"
},
{
  "name": "62F",
```

```

        "dCMark": "C",
        "date": "170331",
        "currency": "CZK",
        "amount": "100,00"
    }
}
}
}

```

D. MongoDB code to store the JSON output

```

MongoClient client =
MongoClients.create("mongodb+srv://emir:12emir34maram@clusterv1.lvzwhgf.mongodb
.net/?retryWrites=true&w=majority");

MongoDatabase db = client.getDatabase("mt_json_v1");

MongoCollection<Document> coll = db.getCollection("mtcoll");

String jsonString = FileUtils.readFileToString(new
File("31MAR17DAILY.json"), "UTF8");

Document doc = Document.parse(jsonString);

coll.insertOne(doc);

```

E. Atms API response in JSON format

```

[{"atmId":"6788","city":"İstanbul","district":"Arnavutköy","address":"SULTANAHM
ET MEYDANI","name":"SULTANAHMET MEYDANI
2","latitude":41.0072681,"longitude":28.979017,"deviceType":"RTM","exchangeAvai
lable":1,"unitCode":949,"quarter":"Boğazköy
İstiklal","provinceCode":34,"disabledPeopleAtm":"H"}, {"atmId":"2447","city":"İs
tanbul","district":"Fatih","address":"SULTANAHMET FATİH TEVFIKHANE SK NO1
34110","name":"SULTANAHMET FOUR
SEASONS","latitude":41.0064642,"longitude":28.9798357,"deviceType":"RTM","excha
ngeAvailable":1,"unitCode":949,"quarter":"Cankurtaran","provinceCode":34,"disab
ledPeopleAtm":"H"}]

```

F. StateInformation API expected response in JSON format

```
{
  "branchInfo": "MERKEZ ŞUBE 00031",
  "usableBalance": "129083.76",
  "startBalance": "128595.71",
  "endBalance": "128595.71",
  "balance": "129083.76",
  "blockBalance": "0.00"
}
```

G. CreditInterestRates API response in JSON format

```
{ "maxTerm":36,"minAmount":5000,"applicationStartDate":"2021-01-23",
  "interestCount":5,"applicationEndDate":"2099-12-31",
  "fundRate":15,"description":"WEB HESAPLAMA ARACI",
  "taxRate":5,"expenseAmount":393.75,"minTerm":1,"maxAmount":75000,
  "creditAmount":40000,"interests":[{"minTerm":1,"maxTerm":6,"rate":1.15},
  {"minTerm":7,"maxTerm":12,"rate":1.12}],
  "interestTerm":0}
```

H. Web application, com.ozu.openbanking.model.JsonMessage

```
package com.ozu.openbanking.model;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.springframework.boot.configurationprocessor.json.JSONObject;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

@Document(collection = "json_message")
@Data
@AllArgsConstructor
@NoArgsConstructor
public class JsonMessage {
    @Id
    private String id;
    private String reference;
    private JSONObject json;
    private String sender;
    private String type;
    //private String json;

    //    public void setJson(String quote){
    //        JSONObject.quote(quote);
    //    }
}
```

I. Web application, com.ozu.openbanking.model.JsonMessageRepository

```
package com.ozu.openbanking.repository;

import com.ozu.openbanking.model.JsonMessage;
import org.bson.types.ObjectId;
```



```

import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public interface JsonMessageRepository extends MongoRepository<JsonMessage,
ObjectId> {
    Optional<JsonMessage> findJsonMessageByReference(String reference);
    Optional<List<JsonMessage>> findJsonMessagesBySender(String sender);
    Optional<List<JsonMessage>> findJsonMessagesByType(String type);
}

```

J. Web application, com.ozu.openbanking.model.JsonMessageService

```

package com.ozu.openbanking.service;

import com.ozu.openbanking.model.JsonMessage;
import com.ozu.openbanking.repository.JsonMessageRepository;
import org.bson.types.ObjectId;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;

@Service
public class JsonMessageService {
    @Autowired
    private JsonMessageRepository jsonMessageRepository;

    public List<JsonMessage> allJson(){
        return jsonMessageRepository.findAll();
    }
    public Optional<JsonMessage> singleJson(ObjectId id){
        return jsonMessageRepository.findById(id);
    }
    public Optional<JsonMessage> jsonByReference(String reference){return
jsonMessageRepository.findJsonMessageByReference(reference);}
    public Optional<List<JsonMessage>> jsonBySender(String sender){return
jsonMessageRepository.findJsonMessagesBySender(sender);}
    public Optional<List<JsonMessage>> jsonByType(String type){return
jsonMessageRepository.findJsonMessagesByType(type);}
}

```

K. Web application, com.ozu.openbanking.model.JsonMessageController

```

package com.ozu.openbanking.controller;

import com.ozu.openbanking.model.JsonMessage;
import com.ozu.openbanking.service.JsonMessageService;
import org.bson.types.ObjectId;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

```

```

import java.io.IOException;
import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/api/v1/json-messages")
public class JsonMessageController {
    @Autowired
    private JsonMessageService jsonMessageService;

    @GetMapping
    public ResponseEntity<List<JsonMessage>> getAllJsonMessages() {
        return new ResponseEntity<>(jsonMessageService.allJson(),
HttpStatus.OK);
    }
    @GetMapping("/{id}")
    public ResponseEntity<Optional<JsonMessage>>
getSingleJsonMessage(@PathVariable ObjectId id) throws IOException {
        return new ResponseEntity<>(jsonMessageService.singleJson(id),
HttpStatus.OK);
    }
    @GetMapping("/reference")
    public ResponseEntity<Optional<JsonMessage>>
getJsonByReference(@RequestParam("reference") String reference) throws
IOException {
        return new
ResponseEntity<>(jsonMessageService.jsonByReference(reference), HttpStatus.OK);
    }
    // @GetMapping("/sender")
    // public ResponseEntity<List<Optional<JsonMessage>>>
getJsonBySender(@RequestParam("sender") String sender) throws IOException {
    // return new
ResponseEntity<>(List<JsonMessage>jsonMessageService.jsonBySender(sender),
HttpStatus.OK);
    // }

}

```

L. Web application, com.ozu.openbanking.model.SwiftMessage

```

package com.ozu.openbanking.model;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import org.bson.types.Binary;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

@Document(collection="swift_messages")
@Data
@AllArgsConstructor
@NoArgsConstructor
public class SwiftMessage {
    @Id
    private String id;
}

```

```

        // private String reference;
        private String type;
        private String body;
        private String sender;

//      public SwiftMessage(String title) {
//          this.reference = title;
//      }
    }
}

```

M. Web application, com.ozu.openbanking.model.SwiftMessageRepository

```

package com.ozu.openbanking.repository;

import com.ozu.openbanking.model.SwiftMessage;
import org.bson.types.ObjectId;
import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

@Repository
public interface SwiftMessageRepository extends MongoRepository<SwiftMessage,
ObjectId> {
    //Optional<SwiftMessage> findSwiftMessageByReference(String reference);
    Optional<List<SwiftMessage>> findAllByType(String type);
    Optional<List<SwiftMessage>> findAllBySender(String sender);
}

```

N. Web application, com.ozu.openbanking.model.SwiftMessageService

```

package com.ozu.openbanking.service;

import com.ozu.openbanking.model.JsonMessage;
import com.ozu.openbanking.model.SwiftMessage;
import com.ozu.openbanking.repository.JsonMessageRepository;
import com.ozu.openbanking.repository.SwiftMessageRepository;
import com.providesoftware.swift.model.mt.AbstractMT;
import com.providesoftware.swift.model.mt.mt9xx.MT940;
import com.providesoftware.swift.model.mt.mt9xx.MT941;
import com.providesoftware.swift.model.mt.mt9xx.MT942;
import org.apache.commons.io.IOUtils;
import org.bson.types.ObjectId;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.configurationprocessor.json.JSONException;
import org.springframework.boot.configurationprocessor.json.JSONObject;
import org.springframework.core.io.Resource;
import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;

import java.io.ByteArrayInputStream;
import java.io.IOException;
import java.util.List;
import java.util.Optional;

```

```

@Service
public class SwiftMessageService {
    @Autowired
    private SwiftMessageRepository swiftMessageRepository;
    @Autowired
    private JsonMessageRepository jsonMessageRepository;
    @Value("classpath:fin/sample.fin")
    private Resource resource;

    public Optional<SwiftMessage> singleSwiftMessage(ObjectId id) throws
IOException {return swiftMessageRepository.findById(id);}
    public List<SwiftMessage> allSwiftMessages(){ return
swiftMessageRepository.findAll();}
    public Optional<List<SwiftMessage>> swiftMessageBySender(String sender)
throws IOException{ return swiftMessageRepository.findAllBySender(sender);}
    public Optional<List<SwiftMessage>> allSwiftMessagesByType(String type){
return swiftMessageRepository.findAllByType(type);}

    public String addSwiftMessage(MultipartFile file) throws IOException,
JSONException {
        SwiftMessage swiftMessage = new SwiftMessage();
        ByteArrayInputStream stream = new
ByteArrayInputStream(file.getBytes());
        String body = IOUtils.toString(stream, "UTF-8");

        com.providesoftware.swift.model.SwiftMessage sm =
com.providesoftware.swift.model.SwiftMessage.parse(body);
        if (sm.isServiceMessage()) {
            sm =
com.providesoftware.swift.model.SwiftMessage.parse(sm.getUnparsedTexts().getAsF
INString());
            body = sm.toString();
        }

        AbstractMT mtMsg = AbstractMT.parse(body);
        swiftMessage.setBody(body);
        swiftMessage.setSender(mtMsg.getSender());
        swiftMessage.setType(mtMsg.getMessageType());
        swiftMessage = swiftMessageRepository.insert(swiftMessage);
        convertToJson(swiftMessage);
        return swiftMessage.getId();
    }

    public JsonMessage convertToJson(SwiftMessage message) throws IOException,
JSONException {
        //Find SwiftMessage by id
        //Optional<SwiftMessage> message= swiftMessageRepository.findById(id);
        JsonMessage jsonMessage = new JsonMessage();

        String sFin = message.getBody();

        com.providesoftware.swift.model.SwiftMessage sm =
com.providesoftware.swift.model.SwiftMessage.parse(sFin);
        if (sm.isServiceMessage()) {
            sm =
com.providesoftware.swift.model.SwiftMessage.parse(sm.getUnparsedTexts().getAsF
INString());
            sFin = sm.toString();
        }
    }
}

```

```

    }
    AbstractMT mtMsg = AbstractMT.parse(sFin);
    jsonMessage.setSender(mtMsg.getSender());
    jsonMessage.setType(mtMsg.getMessageType());

    if (mtMsg.isType(940)) {
        MT940 mt = (MT940) mtMsg;
        jsonMessage.setReference(mt.getField20().getValue());
        JSONObject jsonObject = new JSONObject(mt.toJson());
        jsonMessage.setJson(jsonObject);
    }
    else if (mtMsg.isType(941)) {
        MT941 mt = (MT941) mtMsg;
        jsonMessage.setReference(mt.getField20().getValue());
        JSONObject jsonObject = new JSONObject(mt.toJson());
        jsonMessage.setJson(jsonObject);
    }
    else if (mtMsg.isType(942)) {
        MT942 mt = (MT942) mtMsg;
        jsonMessage.setReference(mt.getField20().getValue());
        JSONObject jsonObject = new JSONObject(mt.toJson());
        jsonMessage.setJson(jsonObject);
    }
    else {
        System.out.println("Unsupported message type.");
    }
    jsonMessageRepository.insert(jsonMessage);
    return jsonMessage;
}

public Optional<List<JsonMessage>> convertAllToJson() throws JSONException,
IOException {
    List<SwiftMessage> swiftMessages= swiftMessageRepository.findAll();
    for(SwiftMessage sm : swiftMessages){
        jsonMessageRepository.insert(convertToJson(sm));
    }
    return Optional.of(jsonMessageRepository.findAll());
}

public void addAllSwiftMessages() throws IOException {
}
}

```

O. Web application, com.ozu.openbanking.model.SwiftMessageController

```

package com.ozu.openbanking.controller;

import com.ozu.openbanking.service.SwiftMessageService;
import com.ozu.openbanking.model.SwiftMessage;
import com.ozu.openbanking.model.JsonMessage;
import org.bson.types.ObjectId;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.configurationprocessor.json.JSONException;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

```

```

import org.springframework.web.multipart.MultipartFile;

import java.io.IOException;
import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/api/v1/swift-messages")
public class SwiftMessageController {
    @Autowired
    private SwiftMessageService swiftMessageService;

    @PostMapping("/add")
    public String addSwiftMessage(@RequestParam("file") MultipartFile file)
    throws IOException, JSONException {
        String id = swiftMessageService.addSwiftMessage(file);
        return "redirect:/swift_messages/"+id;
    }
    @GetMapping
    public ResponseEntity<List<SwiftMessage>> getAllSwiftMessages() {
        return new ResponseEntity<>(swiftMessageService.allSwiftMessages(),
    HttpStatus.OK);
    }
    @GetMapping("/{id}")
    public ResponseEntity<Optional<SwiftMessage>>
    getSingleSwiftMessage(@PathVariable ObjectId id) throws IOException {
        return new
    ResponseEntity<Optional<SwiftMessage>>(swiftMessageService.singleSwiftMessage(i
    d), HttpStatus.OK);
    }
    @GetMapping("/convertAll")
    public ResponseEntity<Optional<List<JsonMessage>>> convertAll() throws
    JSONException, IOException {
        return new
    ResponseEntity<Optional<List<JsonMessage>>>(swiftMessageService.convertAllToJso
    n(), HttpStatus.OK);
    }
}

```

P. Web application, com.ozu.openbanking

```

package com.ozu.openbanking;

import com.ozu.openbanking.storage.StorageProperties;
import com.ozu.openbanking.storage.StorageService;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import
    org.springframework.boot.context.properties.EnableConfigurationProperties;
import org.springframework.context.annotation.Bean;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@SpringBootApplication
@RestController
@EnableConfigurationProperties(StorageProperties.class)

```

```

public class OpenBankingApplication {

    public static void main(String[] args)
    {SpringApplication.run(OpenBankingApplication.class, args);}

}

```

Q. Spring Dependency diagrams

